NOTICE

- Deadlines
  - Programming assignment 2 grades are on Canvas
  - 5/11 (Mon): No class, recorded lecture will be online
  - (5/13 11:59 PM) Programming assignment 3
  - (5/13 11:59 PM) Midterm Quiz 3

SOCKET PROGRAMMING EXAMPLE: MULTI-THREADED SERVER

... omit the code for creating and binding sockets ...

```
if (listen(server_fd, 10) < 0) {
   perror("listen failed");
   exit(EXIT_FAILURE);
}
```

Listen.
Prefer to have ~10 connections
Linux will set it to somaxconn

cat /proc/sys/net/core/somaxconn

```
pthread_t tid;
while (1) {
   if ((csocket = accept(server_fd,
                         (struct sockaddr *)&client,
                         (socklen_t *)&c)) < 0) {
      printf("Server waits for a connection\n");
      continue;
   }
   conn_cnt++
   printf("Server accepts the connection [%d]\n", conn_cnt);
   int *new_sock = malloc(sizeof(int)); *new_sock = csocket;

   if (pthread_create(&tid, NULL, conn_handler, (void *)new_sock)  < 0) {
      perror("Error: cannot start a thread");
      continue;
   }
   pthread_detach(tid);
}

return 0;        // this thread infinitely runs, this line won't be reached
```

Create a threat that handles the communications between this server and the connected client (we pass socket_desc as an arg)

```
void *conn_handler(void *arg) {
  int rlen;
  int socket_desc = *(int *)arg; free(arg);
  char buffer[BUF_SIZE] = { 0 };
  char *ack = "Received";

  while ((rlen = read(socket_desc, buffer, BUF_SIZE)) > 0) {
     buffer[rlen] = '\0';
     if (rlen < 0) continue;

     printf("(server) Received: %s\n", buffer);
     send(socket_desc, ack, strlen(ack), 0);
     printf("(server) Ack sent\n");
  }

  close(socket_desc);
  return 0;
}
```

Keep reading data from the socket
1. If no data, it continues
2. If data is, it prints out the data
3. Then it sends the ack message

SOCKET PROGRAMMING EXAMPLE: CLIENT CONNECTIONS

```
#/bin/bash

for i in $(seq 1 20); do
  (echo "Hello from client $i" | nc -w 3 localhost 8080 && echo "[$i] connected") || echo "[$i] REFUSED" &
done
wait
```

CS 374: Operating Systems I

Part III – Networking I

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

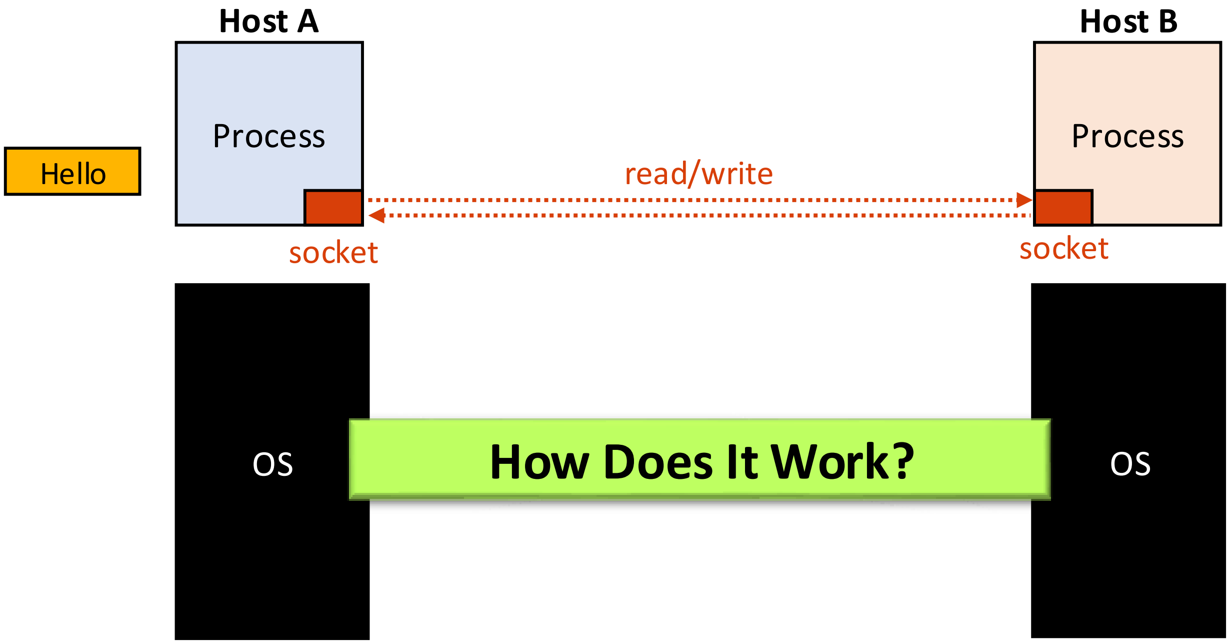
sanghyun.hong@oregonstate.edu

TOPICS FOR TODAY

- Part III: Networking
  - Provide abstraction
    - OSI models (7-layer vs. TCP/IP 4-layer)
    - Packet encapsulation
  - Offer standard interface
    - Socket – covered in the previous lecture
  - Manage resources
    - Packet encapsulation in detail
    - NIC, Ethernet & ARP, IP & Routing, TCP/UDP

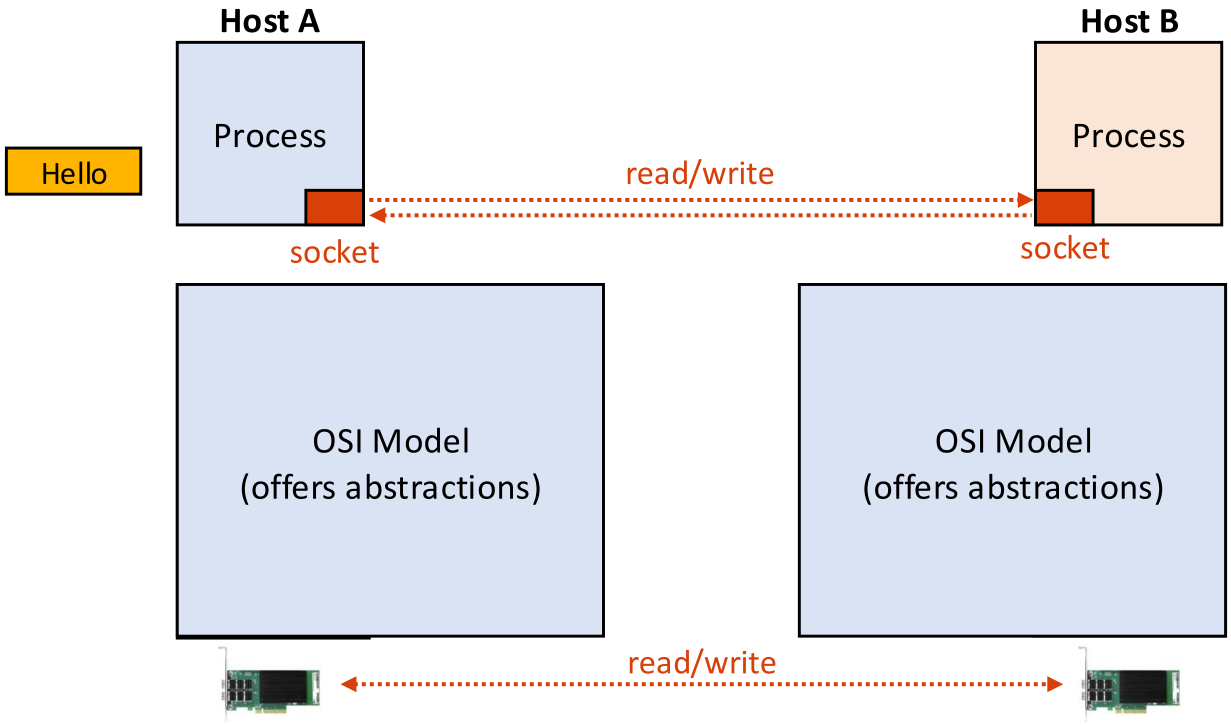
(COMPUTER) NETWORKING

- Networking
  - **Definition:** two or more applications on different computers (hosts) exchanging data



PROVIDE ABSTRACTION

- **Open Systems Interconnection (OSI) model**



PROVIDE ABSTRACTION: 7-LAYER MODEL

- **O**pen **S**ystems **I**nterconnection (**OSI**) model

Host A

Process

Hello

socket

Application: HTTP, FTP
Presentation: TCP, UDP
Session
Transport
Network: IP, Port
Data-link: Ethernet
Physical: Fiber, Wi-Fi

read/write

Host B

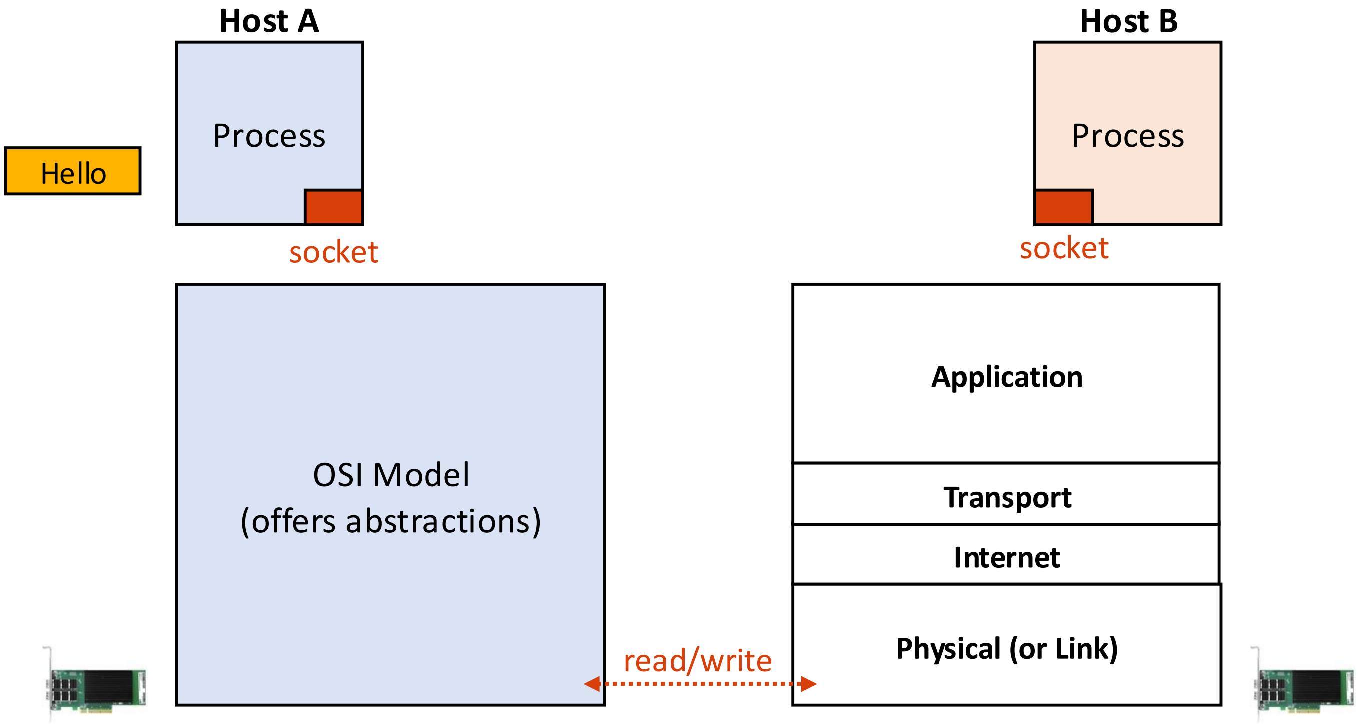
Process

socket

OSI Model
(offers abstractions)

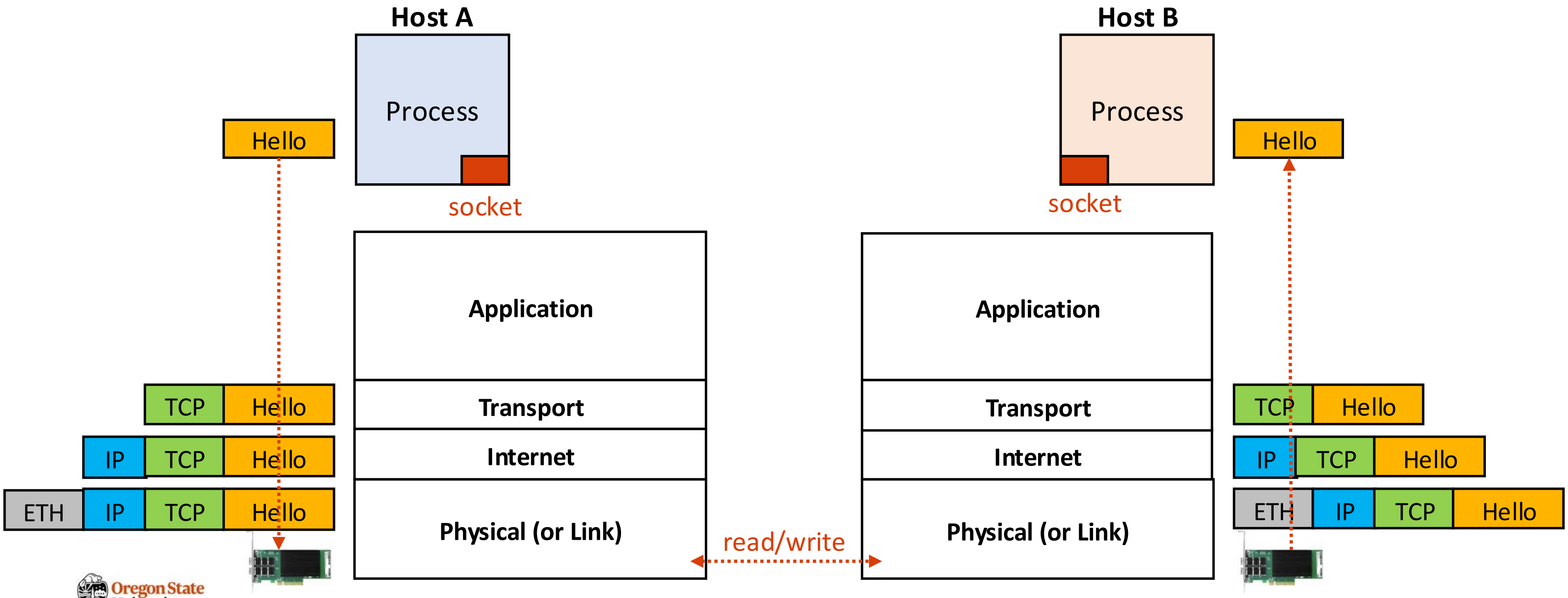
PROVIDE ABSTRACTION: TCP/IP 4-LAYER MODEL

- **O**pen **S**ystems **I**nterconnection (**OSI**) model



PROVIDE ABSTRACTION: PACKET ENCAPSULATION

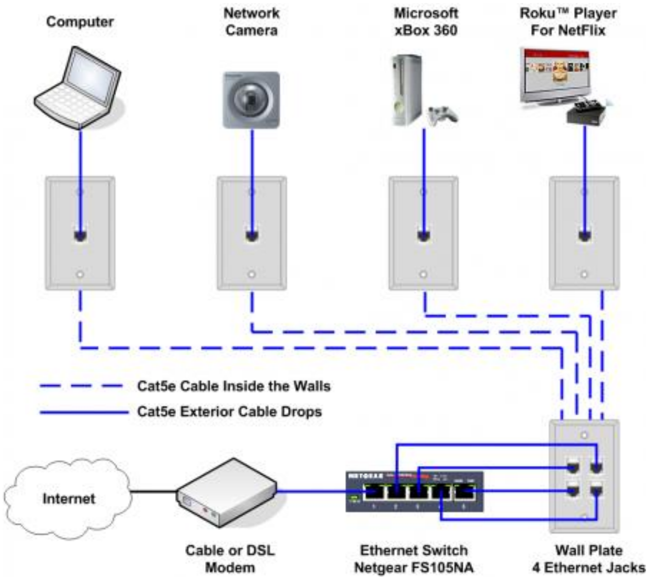
- In the TCP/IP 4-layer model



PROVIDE ABSTRACTION: ETHERNET (PHYSICAL LAYER)

- Ethernet Protocol (~80s)
  - Each network device (NIC) has 48-bit **MAC address**
  - Each NIC is connected via Ethernet **cable**

MAC (7C…) MAC (4D…) MAC (65…) MAC (43…)



MAC (4B 00…)

ETH	IP	TCP	Hello

Physical (or Link)

read/write

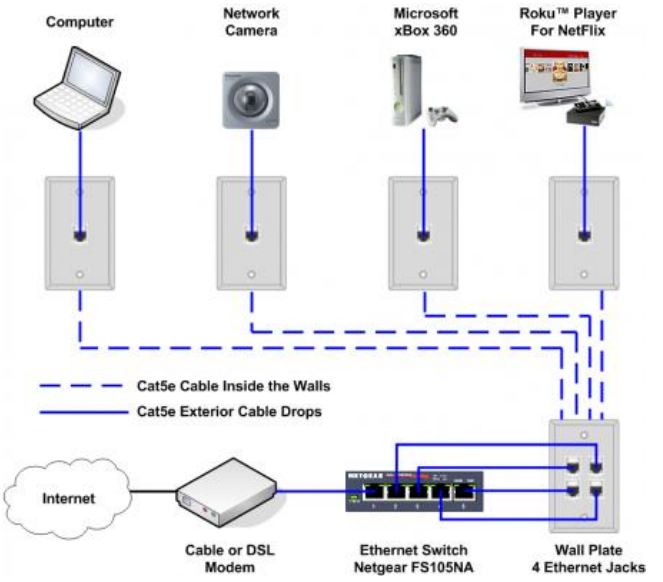
Physical (or Link)

ETH	IP	TCP	Hello

PROVIDE ABSTRACTION: ETHERNET (PHYSICAL LAYER)

- Ethernet Protocol (~80s)
  - Each network device (NIC) has 48-bit **MAC address**
  - Each NIC is connected via Ethernet **cable**
  - ETH **header** contains:
    - (64 bit) Preamble (0x111111111… or a unique data)
    - (48-bit) Destination MAC address
    - (48-bit) Source MAC address
    - (16-bit) Type
    - (up to 1500 bytes) Data
    - (32-bit) CRC for error correcting

MAC (7C…) MAC (4D…) MAC (65…) MAC (43…)



MAC (4B 00…)

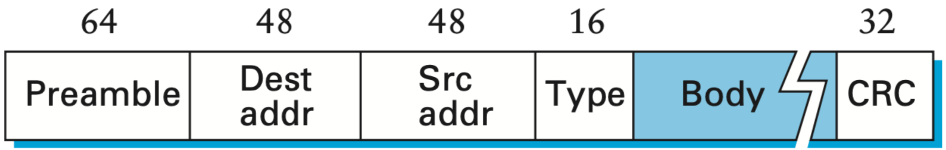
ETH	IP	TCP	Hello

Physical (or Link) ← read/write → **Physical (or Link)**

ETH	IP	TCP	Hello

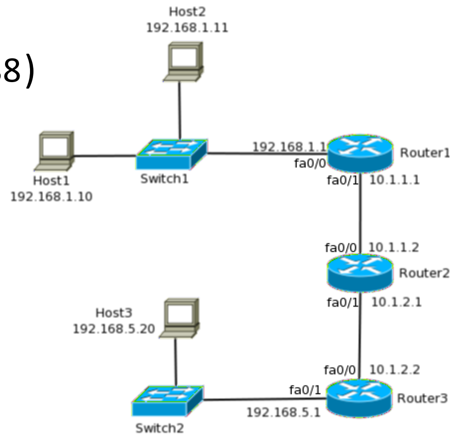
PROVIDE ABSTRACTION: ETHERNET (PHYSICAL LAYER)

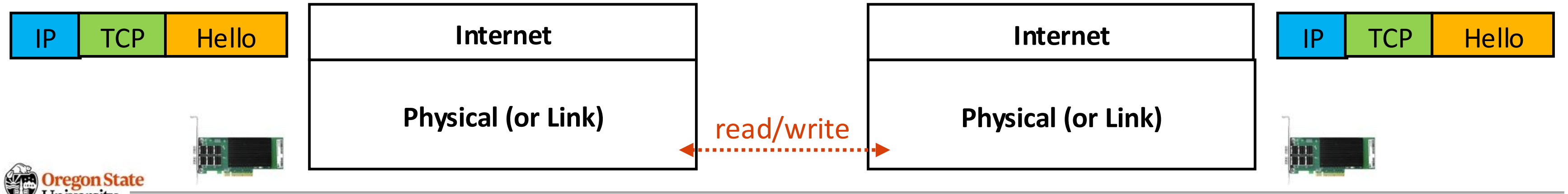
- Ethernet Protocol (~80s)
  - Each network device (NIC) has 48-bit **MAC address**
  - Each NIC is connected via Ethernet **cable**
  - ETH **header** contains:
    - (64 bit) Preamble (0x111111111… or a unique data)
    - (48-bit) Destination MAC address
    - (48-bit) Source MAC address
    - (16-bit) Type
    - (up to 1500 bytes) Data
    - (32-bit) CRC for error correcting



PROVIDE ABSTRACTION: IP LAYER

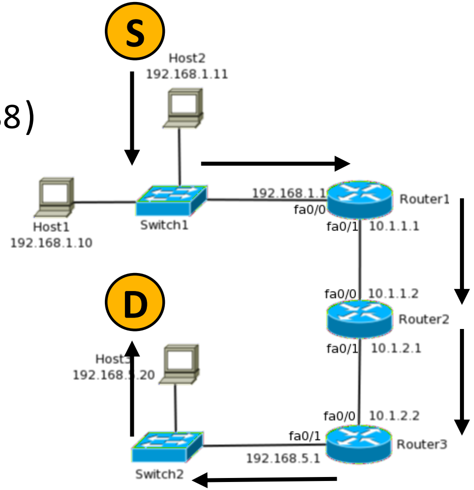
- Internet Protocol (IP)
  - IP allows us to connect multiple networks
  - Each host has a unique IP address
    - **IPv4**: 32-bit address (e.g., 147.56.28.101)
    - **IPv6**: 128-bit address (e.g., 2001:db8:3333:4444:5555:6666:7777:8888)





PROVIDE ABSTRACTION: IP LAYER

- Internet Protocol (IP)
  - IP allows us to connect multiple networks
  - Each host has a unique IP address
    - **IPv4**: 32-bit address (e.g., 147.56.28.101)
    - **IPv6**: 128-bit address (e.g., 2001:db8:3333:4444:5555:6666:7777:8888)
  - IP data (packets) is **routed** based on **destination IP**



IP	TCP	Hello

Internet
Physical (or Link)

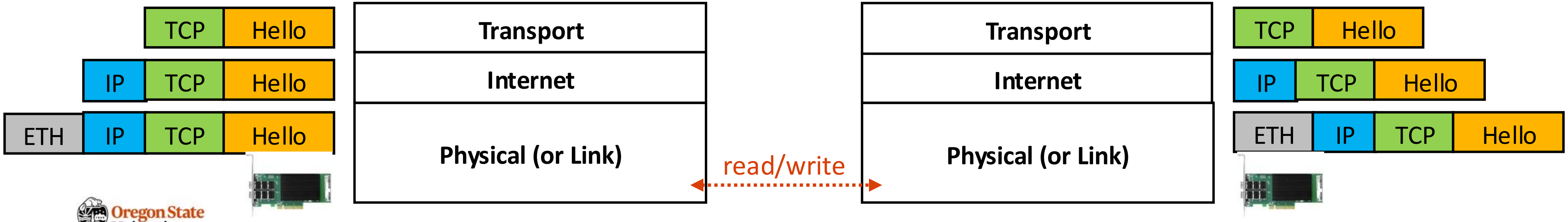
read/write

Internet
Physical (or Link)

IP	TCP	Hello

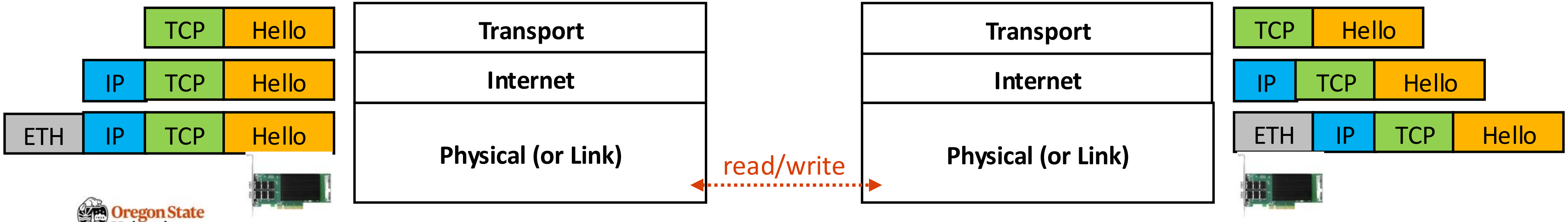
PROVIDE ABSTRACTION: TRANSPORT LAYER

- TCP vs UDP **Protocol**
  - <u>T</u>ransmission <u>C</u>ontrol <u>P</u>rotocol: **<u>TCP</u>** Packet
    - (16-bit, for each) Source and destination ports
    - (32-bit) Sequence number
    - (32-bit) Acknowledgement number
    - Others: flags, checksums, window-size, pointer, …
  - **<u>U</u>**ser **<u>D</u>**atagram **<u>P</u>**rotocol: **<u>UDP</u>** Packet
    - (16-bit, for each) Source and destination port
    - (16-bit, for each) Length and checksum



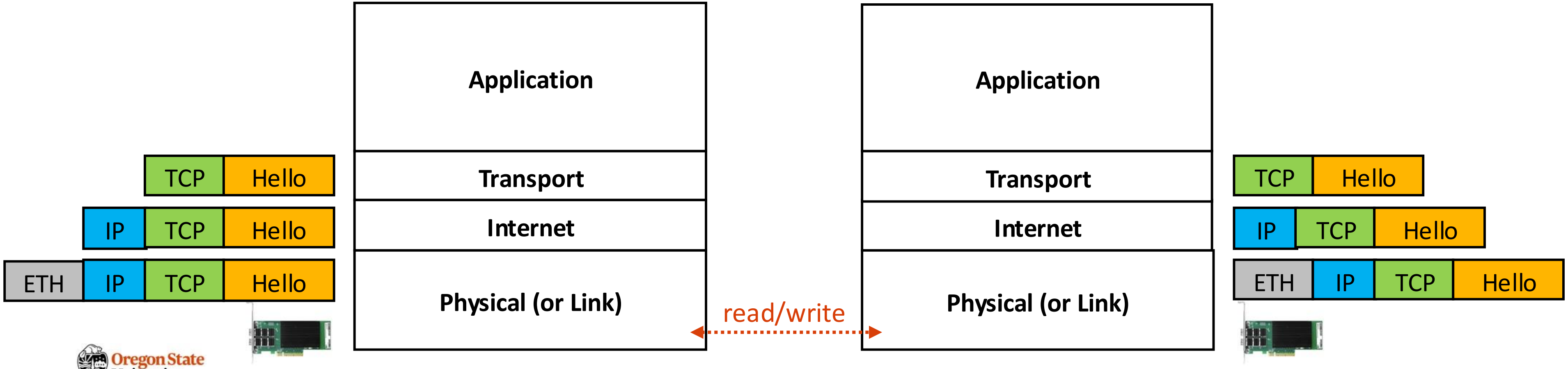
PROVIDE ABSTRACTION: TRANSPORT LAYER

- TCP vs UDP **Protocol**
  - TCP requires an established connection, but UDP is not (broadcast)
  - TCP can use sequences, but UDP is not
  - TCP is like a PIPE; data won't be lost, but UDP will (can lose data)
  - TCP guarantees delivery, but UDP does not
  - TCP is slower than UDP (suppose that we deliver all the packets)



PROVIDE ABSTRACTION: APPLICATION LAYER

- Application layer
  - Support various user-defined or OS-defined protocols (on top of TCP/UDP)
  - **TCP-based :** HTTPS, HTTP, SMTP, POP, FTP, …
  - **UDP-based:** Video streaming, conferencing, DNS, VoIP, …



TOPICS FOR TODAY

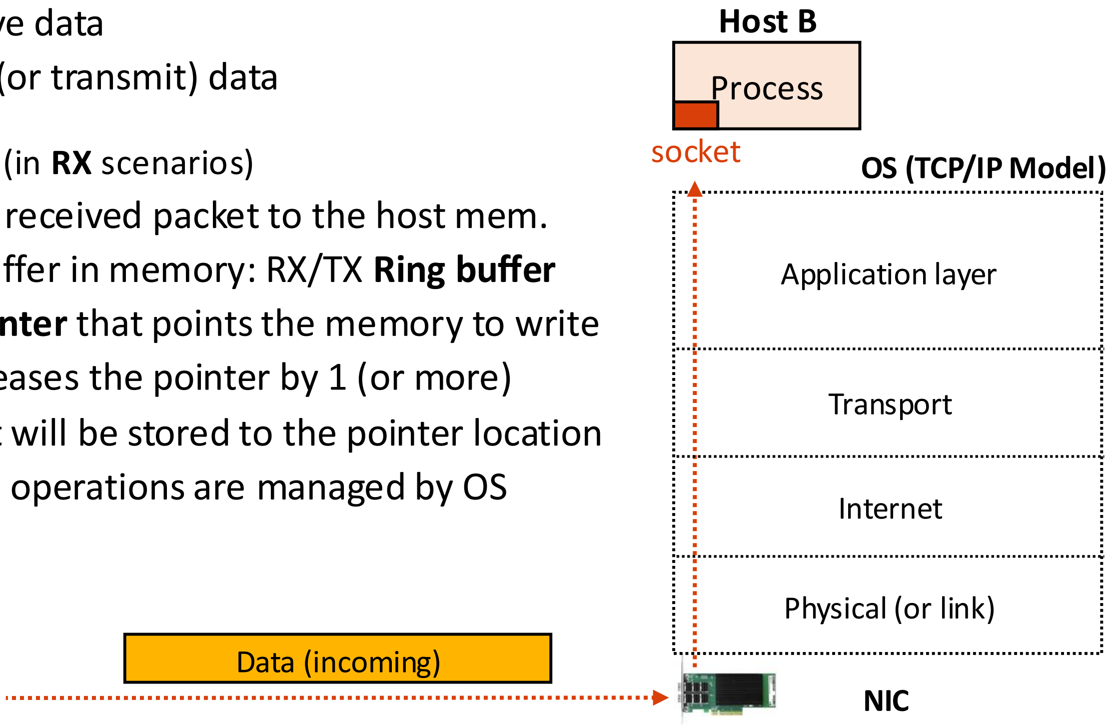
- Part III: Networking
  - Provide abstraction
    - OSI models (7-layer vs. TCP/IP 4-layer)
    - Packet encapsulation
  - Offer standard interface
    - Socket - covered in the previous lecture
  - Manage resources
    - Packet encapsulation in detail
    - NIC, Ethernet & ARP, IP & Routing, TCP/UDP

PACKET ENCAPSULATION IN DETAIL

- Dive into the encapsulation
  - Hardware: NIC (and the network driver)
  - Physical: MAC address-based communication
  - Internet: IP address-based communication
  - Transport: Define protocols, *e.g.*, TCP or UDP
  - Application: Define custom protocols, *e.g.*, HTTP, HTTPS, FTP, etc.

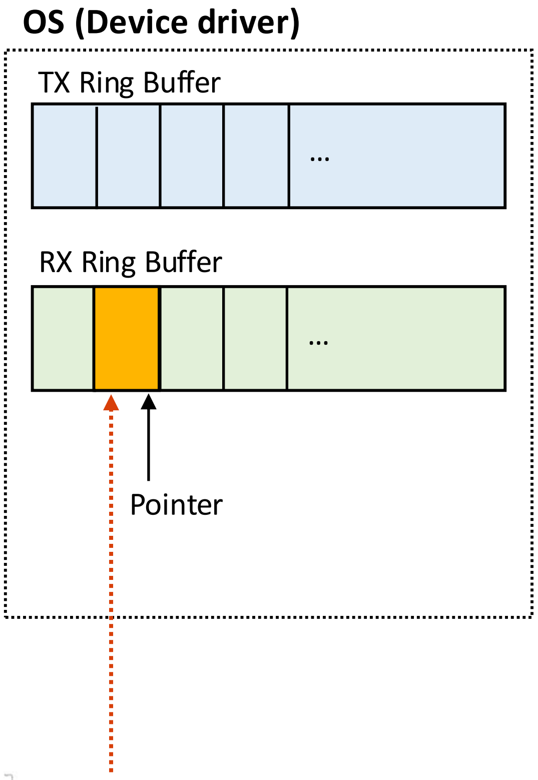
PACKET ENCAPSULATION: NIC

- **<u>N</u>**etwork **<u>I</u>**nterface **<u>C</u>**ard
  - Networking terminology
    - Receive (RX) : receive data
    - Transmit (TX): send (or transmit) data
  - NIC and OS interaction (in **RX** scenarios)
    - First, NIC copies the received packet to the host mem.
      - The OS has a buffer in memory: RX/TX **Ring buffer**
      - It also has **a pointer** that points the memory to write
    - Second, the OS increases the pointer by 1 (or more)
      - The next packet will be stored to the pointer location
    - The first and second operations are managed by OS



PACKET ENCAPSULATION: NIC

- **N**etwork **I**nterface **C**ard
  - Networking terminology
    - Receive (RX) : receive data
    - Transmit (TX): send (or transmit) data
  - NIC and OS interaction (in **RX** scenarios)
    - First, NIC copies the received packet to the host mem.
      - The OS has a buffer in memory: RX/TX **Ring buffer**
      - It also has **a pointer** that points the memory to write
    - Second, the OS increases the pointer by 1 (or more)
      - The next packet will be stored to the pointer location
    - The first and second operations are managed by OS



PACKET ENCAPSULATION: NIC

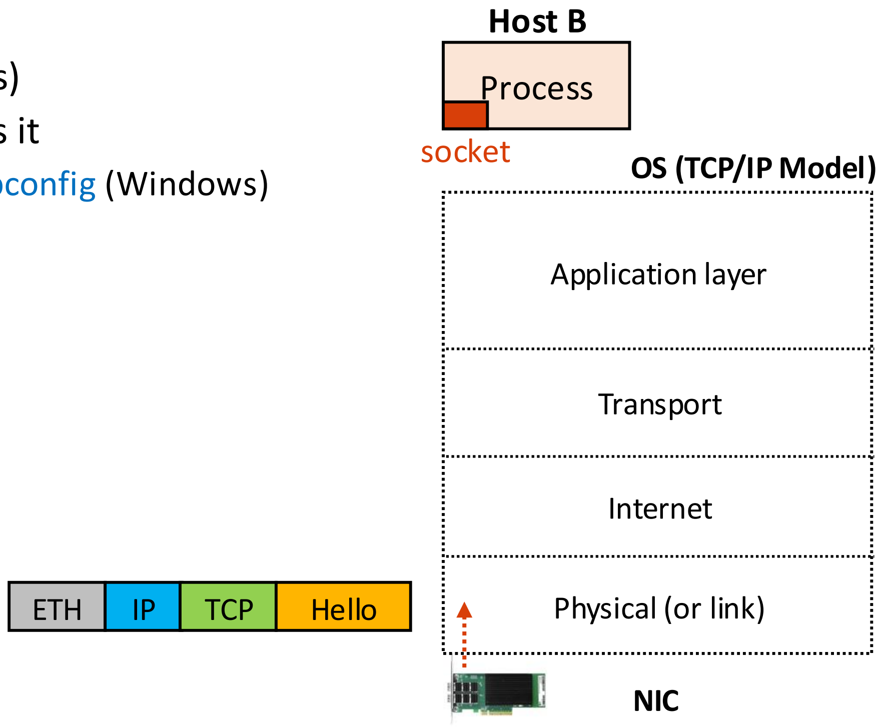
- **N**etwork **I**nterface **C**ard
  - Check the size of RX/TX ring buffer
    - $ ethtool -g eno1np0
    - $ ethtool -G eno1np0 rx 4096
  - NIC statistics
    - $ cat /proc/net/dev

PACKET ENCAPSULATION IN DETAIL

- Dive into the encapsulation
  - Hardware: NIC (and the network driver)
  - Physical: MAC address-based communication
  - Internet: IP address-based communication
  - Transport: Define protocols, *e.g.*, TCP or UDP
  - Application: Define custom protocols, *e.g.*, HTTP, HTTPS, FTP, etc.

PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Windows)



PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Windows)

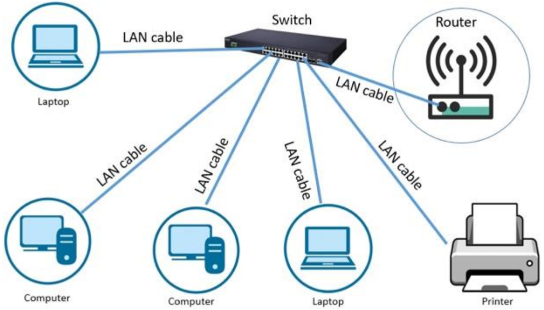
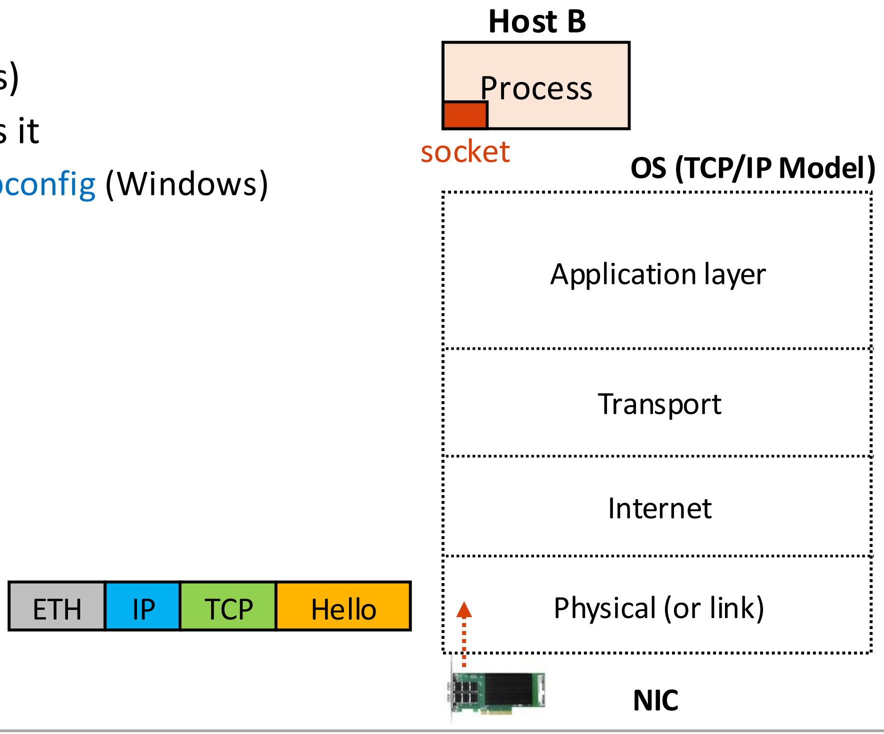
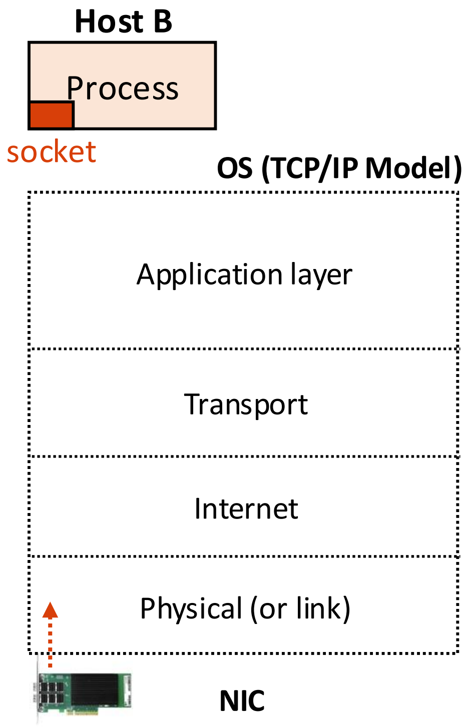


Illustration of a LAN
(Hosts are located quite nearby)



PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Windows)
  - Ethernet packet contains:
    - Preamble (0x111111111… or a unique 64-bit data)
    - Destination MAC address (48-bit)
    - Source MAC address (48-bit)
    - Type (16-bit)
    - Data (variable ~1500 bytes)
    - CRC (32-bit; for error correcting)

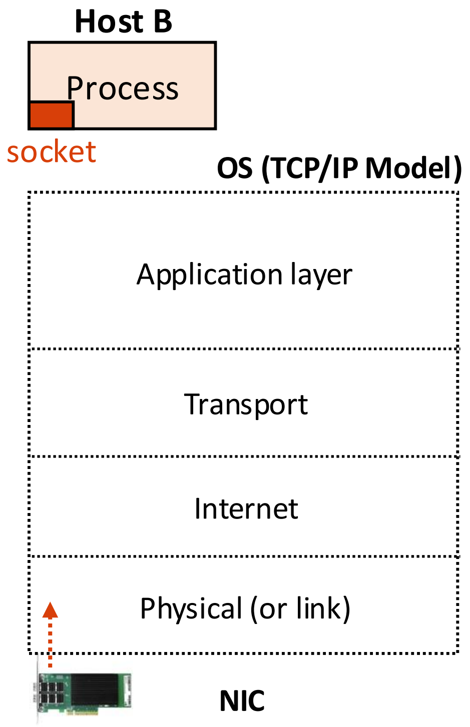


PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Windows)
  - Ethernet packet contains:

NIC Manages

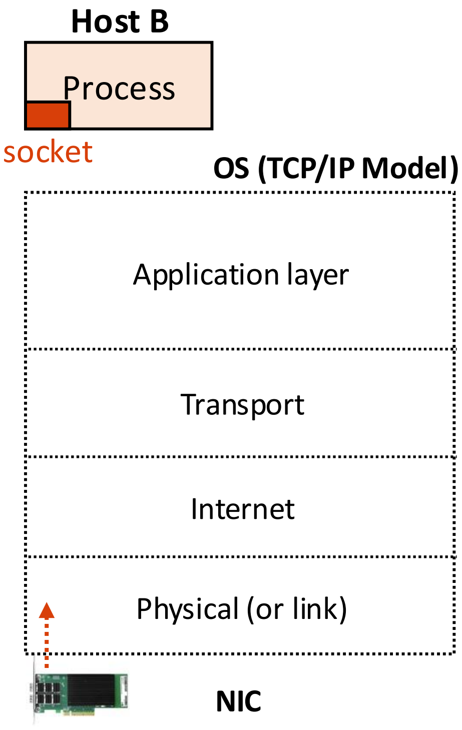
    - Preamble (0x111111111… or unique 64-bit data)
    - Destination MAC address (48-bit)
    - Source MAC address (48-bit)
    - Type (16-bit)
    - Data (variable ~1500 bytes)
    - CRC (32-bit; for error correcting)



PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Windows)
  - Ethernet packet contains:
    - Preamble (0x111111111… or unique 64-bit data)
    - Destination MAC address (48-bit)
    - Source MAC address (48-bit)
    - Type (16-bit)
    - Data (variable ~1500 bytes)
    - CRC (32-bit; for error correcting)

OS Manages



PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Win
  - Ethernet packet contains:
    - Preamble (unique 64-bit data)
    - Destination MAC address (48-bit)
    - Source MAC address (48-bit)
    - Type (16-bit)
    - Data (variable ~1500 bytes)
    - CRC (32-bit; for error correcting)

```
/*
 *      This is an Ethernet frame header.
 */

/* allow libcs like musl to deactivate this, glibc does not implement this. */
#ifndef __UAPI_DEF_ETHHDR
#define __UAPI_DEF_ETHHDR               1
#endif

#if __UAPI_DEF_ETHHDR
struct ethhdr {
        unsigned char   h_dest[ETH_ALEN];       /* destination eth addr */
        unsigned char   h_source[ETH_ALEN];     /* source ether addr    */
        __be16          h_proto;                /* packet type ID field */
} __attribute__((packed));
#endif
```

h_dest: Destination MAC address
h_source: Source MAC address
h_proto: Packet type ID field

ETH_ALEN: 6 bytes (48 bits)
(see the kernel code: include/uapi/linux/if_ether.h)

PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- Physical (or link) layer
  - Ethernet Protocol (ETH)
    - Developed in 80s
    - Used for local area networks (LANs)
    - Use 48-bit MAC addresses; NIC has it
      - DIY check: ifconfig (Linux) and ipconfig (Win
  - Ethernet packet contains:
    - Preamble (unique 64-bit data)
    - Destination MAC address (48-bit)
    - Source MAC address (48-bit)
    - Type (16-bit)
    - Data (variable ~1500 bytes)
    - CRC (32-bit; for error correcting)

```c
/*
 *      This is an Ethernet frame header.
 */

/* allow libcs like musl to deactivate this, glibc does not implement this. */
#ifndef __UAPI_DEF_ETHHDR
#define __UAPI_DEF_ETHHDR               1
#endif

#if __UAPI_DEF_ETHHDR
struct ethhdr {
        unsigned char   h_dest[ETH_ALEN];       /* destination eth addr */
        unsigned char   h_source[ETH_ALEN];     /* source ether addr    */
        __be16          h_proto;                /* packet type ID field */
} __attribute__((packed));
#endif
```

ETH | Data (1500 bytes)

ETH_ETH_DATA_LEN: 1500

(see include/uapi/linux/if_ether.h)

PACKET ENCAPSULATION: PHYSICAL (OR LINK) LAYER

- How to read a MAC address: ioctl()
  - System call
  - Create a socket of any type
  - struct ifreq with the interface name
  - OS kernel fills 6-byte MAC address
  - OS reads it from struct ethhdr

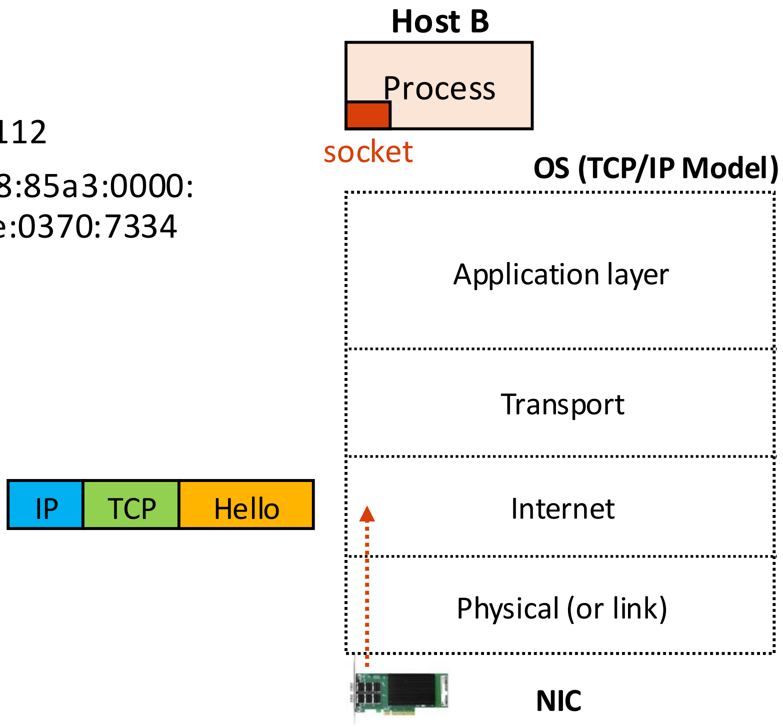
```c
#include <stdio.h>
#include <string.h>
#include <net/if.h>
#include <sys/ioctl.h>
#include <sys/socket.h>

int main() {
   struct ifreq ifr;
   int fd = socket(AF_INET, SOCK_DGRAM, 0);

   strncpy(ifr.ifr_name, "eno1np0", IFNAMSIZ);
   ioctl(fd, SIOCGIFHWADDR, &ifr);

   unsigned char *mac = (unsigned char *)ifr.ifr_hwaddr.sa_data;
   printf("MAC: %02x:%02x:%02x:%02x:%02x:%02x\n",
            mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
   return 0;
}
```

Packet encapsulation in detail

- Dive into the encapsulation
  - Hardware: NIC (and the network driver)
  - Physical: MAC address-based communication
  - Internet: IP address-based communication
  - Transport: Define protocols, *e.g.*, TCP or UDP
  - Application: Define custom protocols, *e.g.*, HTTP, HTTPS, FTP, etc.

PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - Internet Protocol (IP)
    - Use to connect multiple LANs
    - Use IP addresses to locate a host(s)
      - IPv4: 32-bit address, *e.g.*, 54.189.37.112
      - IPv6: 128-bit address, *e.g.*, 2001:0db8:85a3:0000:
        0000:8a2e:0370:7334



Packet Encapsulation: Internet Layer

- Internet Layer
  - Internet Protocol (IP)
    - Use to connect multiple LANs
    - Use IP addresses to locate a host(s)
      - IPv4: 32-bit address, *e.g.*, 54.189.37.112
      - IPv6: 64-bit address, *e.g.*, 2001:0db8:85a3:0000:0000:8a2e:0370:7334

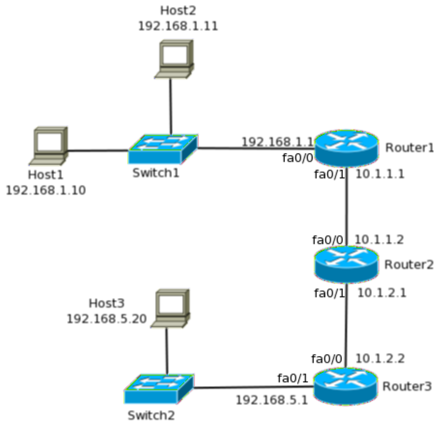
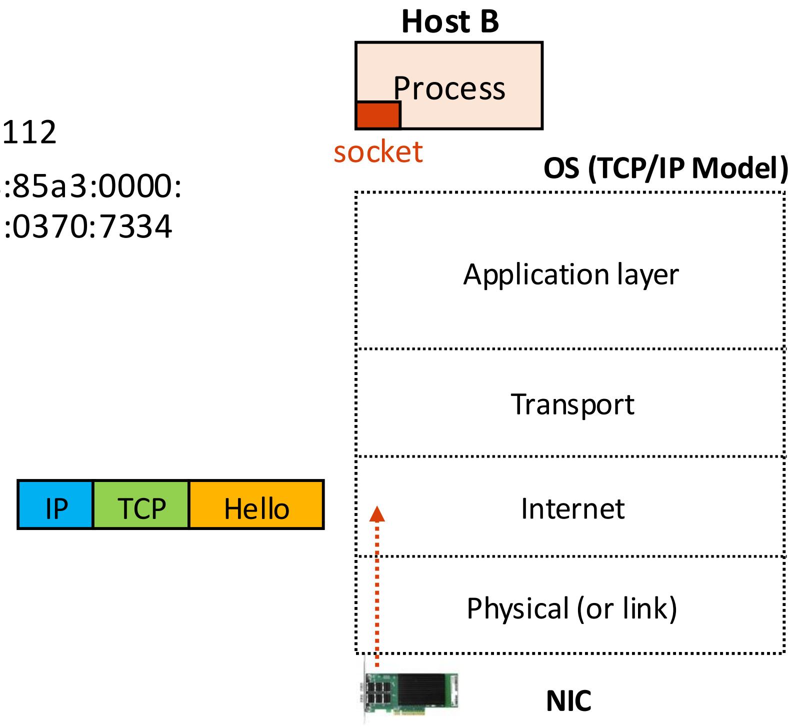
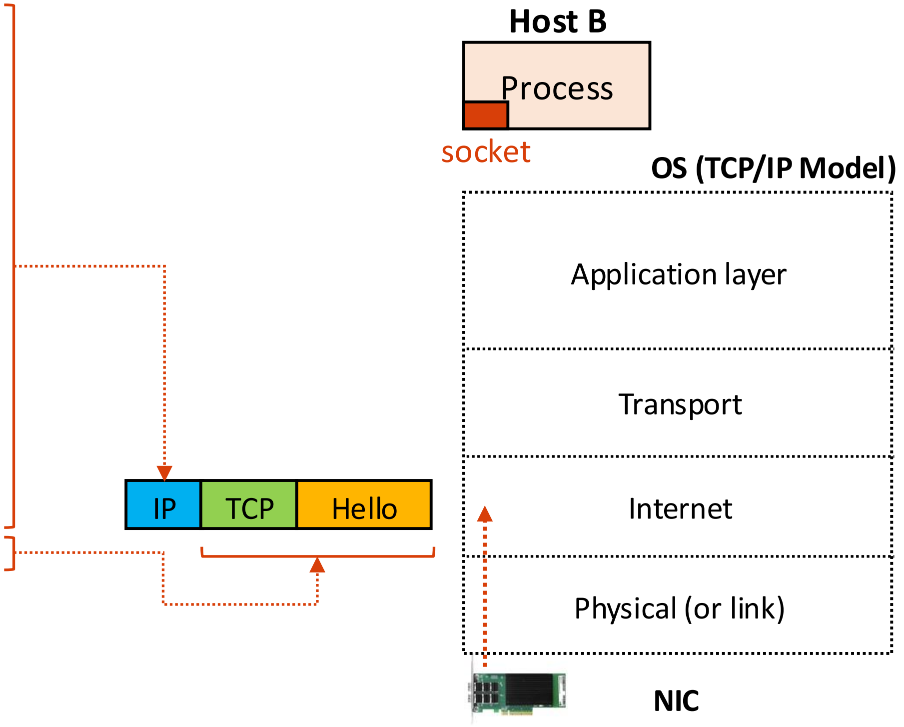


Illustration of the Internet
(Hosts are located remotely)



PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet contains:
    - Version v4/v6 (8-bit)
    - Type of service (8-bit)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)
    - Data (1 ~ 65515 bytes)



PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

IPv4 Header

(source code)

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```

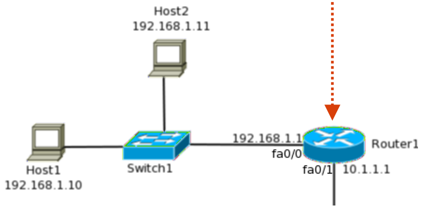
PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

Priority of IP Packet
Important at the router
(CS 372 Topic)

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```



PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

Total packet length
Min : 21 bytes
Max: 65535 bytes

(20 bytes header +
1 ~ 65515 bytes data)

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```

PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

Fragmentation

Oftentimes, we send multiple IP packets that sum up as a whole (ex. the entire data 4000 bytes, we send 4 1000-byte packets)

For example, 3rd IP packet's
ID: 3
Offset: 2000

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```

PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

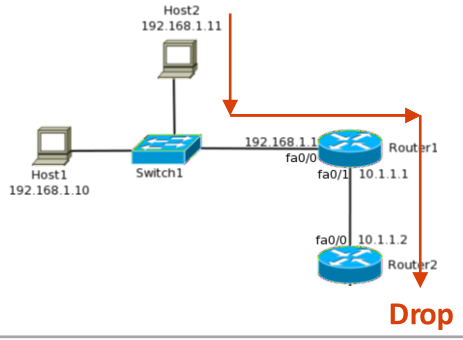
Time-to-live (TTL)
The number of routers a packet can maximally pass (# of "hops")

Reason: drop packets that live too long in a network infrastructure

Example: on the right, setting TTL to 2 -> a packet drops at router 2

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```



PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

IP Protocols

Examples:

TCP	6
UDP	17
ETHERNET	143

(see the list of IP protocols: link)

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```

PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

Checksum
The number to check the *integrity* of a packet while routing.

To make sure the packet is not manipulated, each router re-computes it and compares the checksum value with the stored one. (Q: what's the issue?)

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```

PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - IP packet header contains:
    - Version v4 (8-bit)
    - Type of service (8-bit: reference)
    - Total length (16-bit)
    - Identification (16-bit)
    - Frame offset (16-bit)
    - Time to live (8-bit)
    - Protocol (8-bit)
    - CRC (checksum) (16-bit)
    - Source IP address (32-bit)
    - Destination IP address (32-bit)

IPv4 Header

```
struct iphdr {
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u8    ihl:4,
                version:4;
#elif defined (__BIG_ENDIAN_BITFIELD)
        __u8    version:4,
                ihl:4;
#else
#error  "Please fix <asm/byteorder.h>"
#endif
        __u8    tos;
        __be16  tot_len;
        __be16  id;
        __be16  frag_off;
        __u8    ttl;
        __u8    protocol;
        __sum16 check;
        __be32  saddr;
        __be32  daddr;
        /*The options start here. */
};
```

Source / Destination IPs

Example:

Source : 53.59.125.98
Destination: 22.156.71.44

Router: each router has a routing table that stores the subnetwork addresses. It uses the subnetwork addr and source/destination IPs to decide where to send the packet

PACKET ENCAPSULATION: INTERNET LAYER

- How to read an IP address
  - SOCK_RAW (root privilege required)
  - Create a raw socket
  - Access the struct iphdr as-is
  - ip->saddr/daddr: 32-bit IP address
  - ip->ttl : # hops remaining
  - ip->protocol : TCP=6, UDP=17

```c
#include <stdio.h>
#include <string.h>
#include <netinet/ip.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
   struct ifreq ifr;
   int fd = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);

   while (1) {
      int len = recv(sock, buf, sizeof(buf), 0);
      struct iphdr *ip = (struct iphdr *) buf;

      printf("src: %-16s dst: %-16s TTL: %d proto: %d\n",
            inet_ntoa(*(struct in_addr *)&ip->saddr),
            inet_ntoa(*(struct in_addr *)&ip->daddr),
            ip->ttl, ip->protocol);
   }

   return 0;
}
```

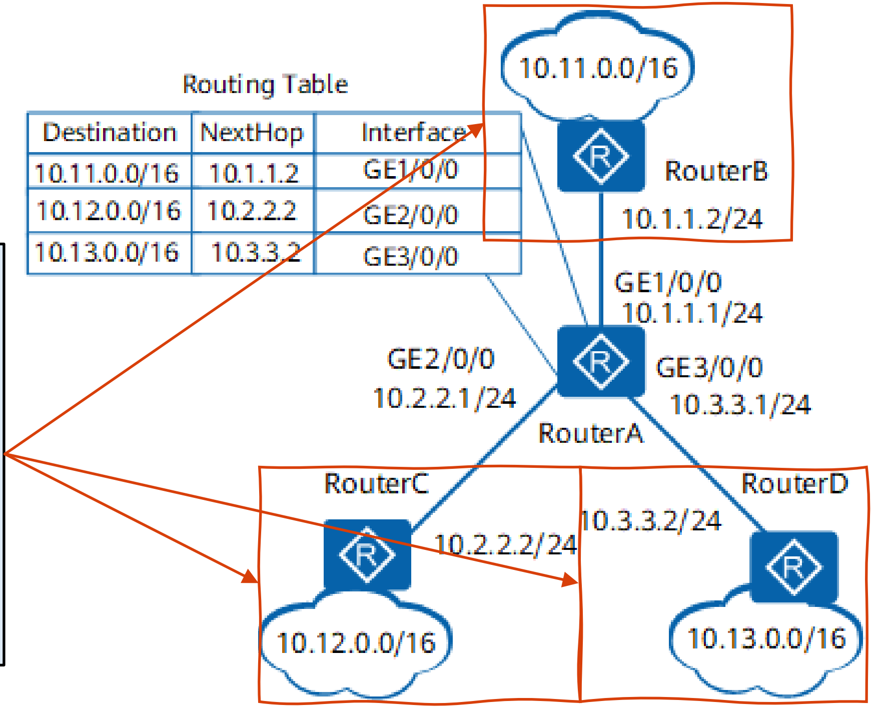
PACKET ENCAPSULATION: INTERNET LAYER

- Internet Layer
  - Router mechanism:
    - Router connects subnetworks
    - **Subnet:** a logical *division* of an IP net

Subnetwork examples:

Router B: 10.11.0.0/16 (65536 addr.)
Router C: 10.12.0.0/16 (same)
Router D: 10.13.0.0/16 (same)

CIDR Notations:
10.11.0.0 : base address
16 : # of leading bits we fix
10.11.0.0/16: 10.11.0.0 – 10.11.255.255



Destination	NextHop	Interface
10.11.0.0/16	10.1.1.2	GE1/0/0
10.12.0.0/16	10.2.2.2	GE2/0/0
10.13.0.0/16	10.3.3.2	GE3/0/0

PACKET ENCAPSULATION: INTERNET LAYER

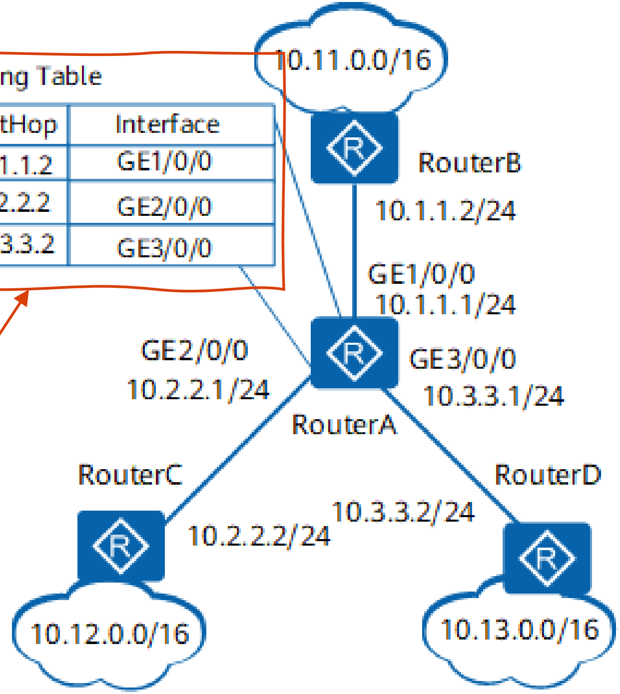
- Internet Layer
  - Router mechanism:
    - Router connects subnetworks
    - **Subnet:** a logical *division* of an IP net
  - Routing table:
    - Describe the network destinations

In Router A: Suppose a packet
comes from the source (10.12.1.45)
moves to the destination (10.11.5.97)

Routing:
Host (src.) -> Router C
Router C -> Router A
Router A -> Router B (subnet: 10.11.0.0/16)
Router B -> Host (dest.)

Routing Table

Destination	NextHop	Interface
10.11.0.0/16	10.1.1.2	GE1/0/0
10.12.0.0/16	10.2.2.2	GE2/0/0
10.13.0.0/16	10.3.3.2	GE3/0/0

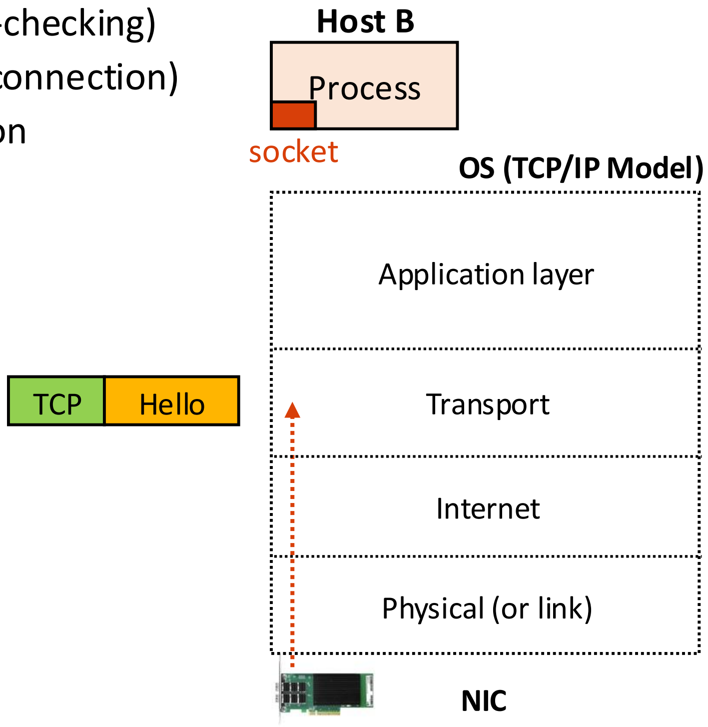


TOPICS FOR TODAY

- Dive into the encapsulation
  - Hardware: NIC (and the network driver)
  - Physical: MAC address-based communication
  - Internet: IP address-based communication
  - Transport: Define protocols, *e.g.*, TCP or UDP
  - Application: Define custom protocols, *e.g.*, HTTP, HTTPS, FTP, etc.

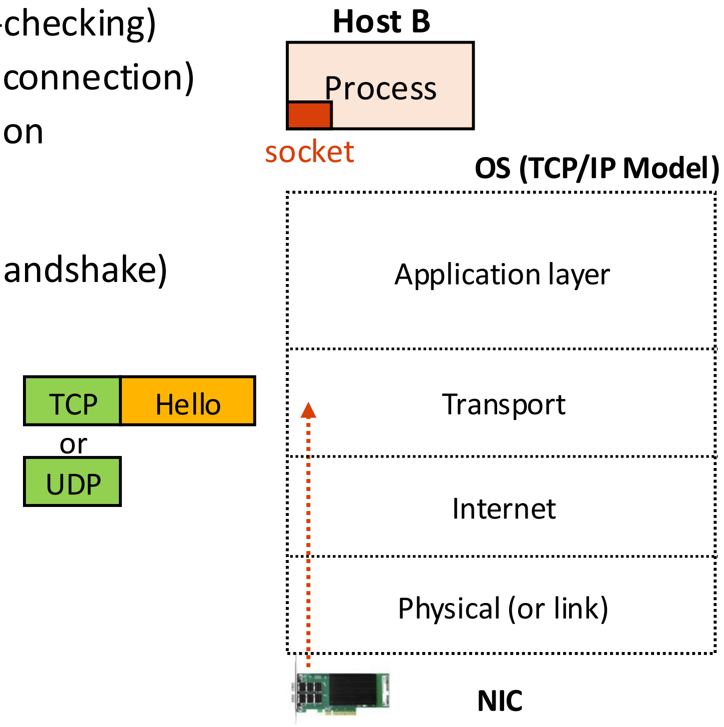
PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - Transmission Control Protocol (TCP)
    - Reliable communications (ordered and error-checking)
    - Connection oriented (first need to establish connection)
    - 3-way handshake for establishing a connection



PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - Transmission Control Protocol (**TCP**)
    - Reliable communication (ordered and error-checking)
    - Connection oriented (first need to establish connection)
    - 3-way handshake for establishing a connection
  - User Datagram Protocol (**UDP**)
    - Simple connectionless communication (no handshake)
    - Less reliable communication (no order)
    - Useful for video/audio streaming [where losing packets is acceptable]



PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - Transmission Control Protocol (TCP)
    - Reliable communications (ordered and error-checking)
    - Connection oriented (first need to establish connection)
    - 3-way handshake for establishing a connection
  - TCP packet header contains:
    - Source ports (16-bit)
    - Destination ports (16-bits)
    - Sequence number (32-bit)
    - Acknowledgement number (32-bit)
    - Others (flags, checksums, window-size, pointer, …)

TCP Header

(source code)

```
struct tcphdr {
        __be16  source;
        __be16  dest;
        __be32  seq;
        __be32  ack_seq;
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u16   res1:4,
                doff:4,
                fin:1,
                syn:1,
                rst:1,
                psh:1,
                ack:1,
                urg:1,
                ece:1,
                cwr:1;
#elif defined(__BIG_ENDIAN_BITFIELD)
        __u16   doff:4,
                res1:4,
                cwr:1,
                ece:1,
                urg:1,
                ack:1,
                psh:1,
                rst:1,
                syn:1,
                fin:1;
#else
#error  "Adjust your <asm/byteorder.h> defines"
#endif
        __be16  window;
        __sum16 check;
        __be16  urg_ptr;
};
```

PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - Transmission Control Protocol (TCP)
    - Reliable communications (ordered and error-checking)
    - Connection oriented (first need to establish connection)
    - 3-way handshake for establishing a connection
  - TCP packet header contains:
    - Source ports (16-bit)
    - Destination ports (16-bits)
    -
    -
    -

Source / Destination Ports

IP addresses are in the IP packet header
TCP header only contains the port numbers (src/dest)

TCP Header

(source code)

```
struct tcphdr {
        __be16  source;
        __be16  dest;
        __be32  seq;
        __be32  ack_seq;
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u16   res1:4,
                doff:4,
                fin:1,
                syn:1,
                rst:1,
                psh:1,
                ack:1,
                urg:1,
                ece:1,
                cwr:1;
#elif defined(__BIG_ENDIAN_BITFIELD)
        __u16   doff:4,
                res1:4,
                cwr:1,
                ece:1,
                urg:1,
                ack:1,
                psh:1,
                rst:1,
                syn:1,
                fin:1;
#else
#error  "Adjust your <asm/byteorder.h> defines"
#endif
        __be16  window;
        __sum16 check;
        __be16  urg_ptr;
};
```

PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - Transmission Control Protocol (TCP)
    - Reliable communications (ordered and error-checking)
    - Connection oriented (first need to establish connection)
    - 3-way handshake for establishing a connection
  - TCP packet header contains:
    - Source ports (16-bit)
    - Destination ports (16-bits)
    - Sequence number (32-bit)
    - Acknowledgement number (32-bit)
    -

Sequence number
The **byte index** of the data a packet has (**4103** / 11945)

Ack number
Receiver: # of packets remaining to receive
Sender : packet # to send next (to the receiver)

TCP Header
(source code)

```
struct tcphdr {
        __be16  source;
        __be16  dest;
        __be32  seq;
        __be32  ack_seq;
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u16   res1:4,
                doff:4,
                fin:1,
                syn:1,
                rst:1,
                psh:1,
                ack:1,
                urg:1,
                ece:1,
                cwr:1;
#elif defined(__BIG_ENDIAN_BITFIELD)
        __u16   doff:4,
                res1:4,
                cwr:1,
                ece:1,
                urg:1,
                ack:1,
                psh:1,
                rst:1,
                syn:1,
                fin:1;
#else
#error  "Adjust your <asm/byteorder.h> defines"
#endif
        __be16  window;
        __sum16 check;
        __be16  urg_ptr;
};
```

PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - Transmission Control Protocol (TCP)
    - Reliable communications (ordered and error-checking)
    - Connection oriented (first need to establish connection)
    - 3-way handshake for establishing a connection
  - TCP packet header contains:
    - Source ports (16-bit)
    - Destination ports (16-bits)
    - Sequence number (32-bit)
    - Acknowledgement number (32-bit)
    - Others (flags, checksums, window-size, pointer, …)

Other fields
Refer to this Wikipedia article (link)

TCP Header
(source code)

```
struct tcphdr {
        __be16  source;
        __be16  dest;
        __be32  seq;
        __be32  ack_seq;
#if defined(__LITTLE_ENDIAN_BITFIELD)
        __u16   res1:4,
                doff:4,
                fin:1,
                syn:1,
                rst:1,
                psh:1,
                ack:1,
                urg:1,
                ece:1,
                cwr:1;
#elif defined(__BIG_ENDIAN_BITFIELD)
        __u16   doff:4,
                res1:4,
                cwr:1,
                ece:1,
                urg:1,
                ack:1,
                psh:1,
                rst:1,
                syn:1,
                fin:1;
#else
#error  "Adjust your <asm/byteorder.h> defines"
#endif
        __be16  window;
        __sum16 check;
        __be16  urg_ptr;
};
```

PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - User Datagram Protocol (UDP)
    - Simple connectionless communication (no handshake)
    - Less reliable communication (no order)
    - Useful for video/audio streaming
  - UDP packet header contains:
    - Source port (16-bit)
    - Destination port (16-bits)
    -
    -

Source / Destination Ports

IP addresses are in the IP packet header
UDP header only contains the port numbers like TCP

```
struct udphdr {
    __be16  source;
    __be16  dest;
    __be16  len;
    __sum16 check;
};
```

PACKET ENCAPSULATION: TRANSPORT LAYER

- Transport layer
  - User Datagram Protocol (UDP)
    - Simple connectionless communication (no handshake)
    - Less reliable communication (no order)
    - Useful for video/audio streaming
  - UDP packet header contains:
    - Source port (16-bit)
    - Destination port (16-bit)
    - Packet length (16-bit)
    - Checksum (16-bits)

Length
Total UDP packet size (max. 65515 bytes)
Note: 65535 – 8-byte UDP header – 20-byte IP header

Checksum
Optional field as IP header already contains this data

```
struct udphdr {
        __be16  source;
        __be16  dest;
        __be16  len;
        __sum16 check;
};
```

PACKET ENCAPSULATION IN DETAIL

- Dive into the encapsulation
  - Hardware: NIC (and the network driver)
  - Physical: MAC address-based communication
  - Internet: IP address-based communication
  - Transport: Define protocols, *e.g.*, TCP or UDP
  - Application: Define custom protocols, *e.g.*, HTTP, HTTPS, FTP, etc. (**CS 372 Topic – Have more fun in this class**)

TOPICS FOR TODAY

- Part III: Networking
  - Provide abstraction
    - OSI models
    - Packet encapsulation
  - Offer standard interface
    - RPC mechanisms (e.g., sockets)
  - Manage resources
    - Packet encapsulation in detail

Thank You!

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu

Oregon State University

TRUE AI
Trustworthy and Responsible AI